МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Департамент цифровых, робототехнических систем и электроники

«Основы работы с библиотекой matplotlib»
Отчет по лабораторной работе № 3
по дисциплине «Искусственный интеллект и машинное обучение»
Вариант 5

Выполнил студент группы ИВТ-б-о-24-1
Грабарь Артемий Павлович
«__» апреля 2026г.
Подпись студента_____
Работа защищена « »_____20__г.
Проверил Воронкин Р.А. _____
(подпись)

Ставрополь 2026

Цель работы: Исследовать базовые возможности библиотеки matplotlib языка программирования Python.

Ссылка на GitHub: https://github.com/Arhi258/Laba_ML_3.git

Задание:

1. Создать общедоступный репозиторий на GitHub, в котором будет использована лицензия MIT и выбранный язык программирования.
2. Выполнить клонирование созданного репозитория на рабочий компьютер.
3. Дополнить файл .gitignore необходимыми правилами для выбранного языка программирования, интерактивной оболочки Jupyter notebook и интегрированной среды разработки.
4. Проработать примеры практической работы.
5. Решить практические задания работы:

Задание 1. Напишите код, который строит график функции y = x2 на интервале [-10, 10]. Добавьте заголовок, подписи осей и сетку.

Задание 2. Постройте три линии на одном графике: y = x (синяя, пунктирная линия), $y = x^2$ (зеленая, штрих-пунктирная линия), $y = x^3$ (красная, сплошная линия). Добавьте легенду и сделайте оси одинакового масштаба.

Задание 3. Сгенерируйте 50 случайных точек и постройте диаграмму рассеяния (scatter plot), где цвет точек зависит от их координаты по оси x, а размер точек зависит от координаты по оси y.

Задание 4. Сгенерируйте 1000 случайных чисел из нормального распределения с параметрами p = 0, 0 = 1 и постройте их гистограмму с 30 бинами. Добавьте вертикальную линию в среднем значении.

Задание 5. Создайте столбчатую диаграмму, которая показывает количество студентов, получивших оценки: "Отлично" - 20 человек, "Хорошо" - 35 человек, "Удовлетворительно" - 30 человек, "Неудовлетворительно" - 15 человек. Добавьте подписи к осям и заголовок.

Задание 6. Используя данные предыдущей задачи, постройте круговую диаграмму с процентными подписями секторов.

Задание 7. Используя mpl_toolkits.mplot3d , постройте 3D-график функции z = $\sin(\sqrt{x^2 + y^2})$ на сетке значений x, y в диапазоне [-5,5].

Задание 8. Постройте четыре графика в одной фигуре (2 x 2 сетка): Линейный график y = x, Парабола y = x^2, Синус y = sin(x), Косинус y = cos(x). Добавьте заголовки к каждому подграфику.

Задание 9. Создайте случайную матрицу 10 x 10 с элементами от 0 до 1 и визуализируйте её как тепловую карту с цветовой шкалой.

Задание 10. На протяжении года измерялся уровень воды в озере: Месяцы: ['Янв', 'Фев', 'Март', 'Апр', 'Май', 'Июнь', 'Июль', 'Авг', 'Сен', 'Окт', 'Нояб', 'Дек']; Уровень воды (м): [2.3, 2.4, 2.6, 2.8, 3.0, 3.1, 3.2, 3.1, 3.0, 2.8, 2.6, 2.4]. Используйте синий цвет линии и добавьте точки с маркерами.

Задание 11. Средняя температура в разные сезоны: Сезоны: [Зима', 'Весна', 'Лето', 'Осень']; Температура (C): [-5, 10, 25, 12]. Используйте разные цвета для каждого сезона.

Задание 12. В каждой задаче требуется: Построить график подинтегральной функции. Вычислить площадь под кривой на заданном отрезке как значение определенного интеграла. Закрасить область под графиком, чтобы визуализировать интеграл. Найдите площадь под функцией: f(x) = 1/x на интервале [1, 5]. Дополнительные требования: В каждой задаче используйте функцию fill_between() для закрашивания площади под кривой. Добавьте подписи осей и заголовок. При необходимости, отобразите оси X и Y для лучшей визуализации.

Задание 13. Во всех задачах требуется: Построить трехмерный график функции f(x,y) в заданных пределах. Использовать библиотеку Matplotlib для визуализации. Оформить график: добавить заголовок, подписи осей и цветовую карту (если уместно). Построить косинус: f(x, y) = $\sqrt{x^2+y^2}$ на интервале x, y Є [-5, 5]. Дополнительные требования: Используйте plot_surface() для плотных поверхностей. Используйте plot_wireframe() для

каркасных моделей. Настроите цветовую карту через стар. Добавьте подписи осей и заголовки.

6. Зафиксировать сделанные изменения в репозитории.
7. Отправить сделанные изменения на сервер GitHub.

Ход работы:

Был создан общедоступный репозиторий на GitHub, в котором будет использована лицензия MIT.

Было выполнено клонирование данного репозитория на рабочий компьютер.

Файл .gitignore был дополнен необходимыми правилами для работы с Anaconda. Был изменен файл README.

Было выполнено задание 1. С помощью языка Python и библиотек NumPy и Matplotlib был построен график квадратичной функции $y = x^2$. В коде задан диапазон значений x от -10 до 10, сформирован массив соответствующих значений y и настроена визуализация: добавлены подписи осей, заголовок и координатная сетка. Результат выполнения задания представлен на рисунке 1.

```
[1]: import matplotlib.pyplot as plt
     import numpy as np

[2]: x = np.linspace(-10, 10, 100)

[3]: y = x**2

[7]: plt.plot(x, y)
     plt.title("График функции y = x^2")
     plt.xlabel("x")
     plt.ylabel("y")
     plt.grid(True)
     plt.show()
```

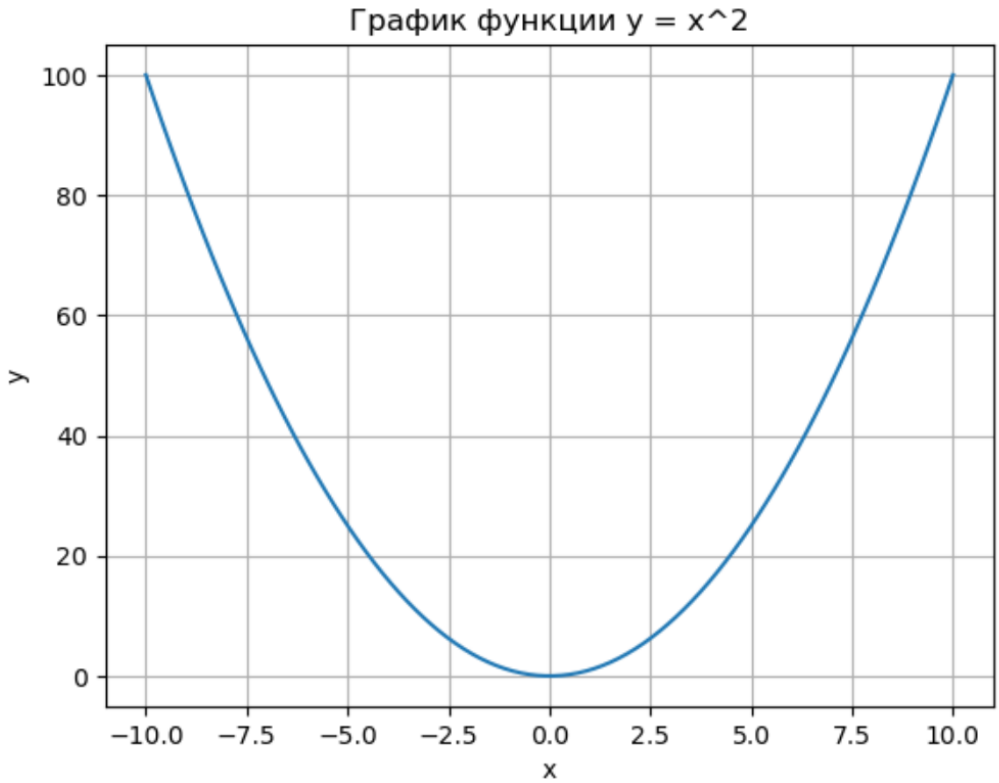

Рисунок 1. Результат выполнения задания 1

Было выполнено задание 2. С помощью библиотек NumPy и Matplotlib на одном холсте были построены графики трех функций: линейной, квадратичной и кубической. Каждая кривая имеет уникальный цвет и стиль линии для удобства идентификации. Также в коде реализована настройка равного масштаба осей, добавлена легенда, координатная сетка и соответствующие подписи. Результат выполнения задания представлен на рисунке 2.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(-3, 3, 100)
```

```
plt.figure(figsize=(7, 7))

plt.plot(x, x, color='blue', linestyle='--', label='y = x')
plt.plot(x, x**2, color='green', linestyle='-.', label='y = x^2')
plt.plot(x, x**3, color='red', linestyle='-', label='y = x^3')
plt.axis('equal')

plt.legend()
plt.grid(True)
plt.title("Три линии на одном графике")
plt.xlabel("x")
plt.ylabel("y")

plt.show()
```

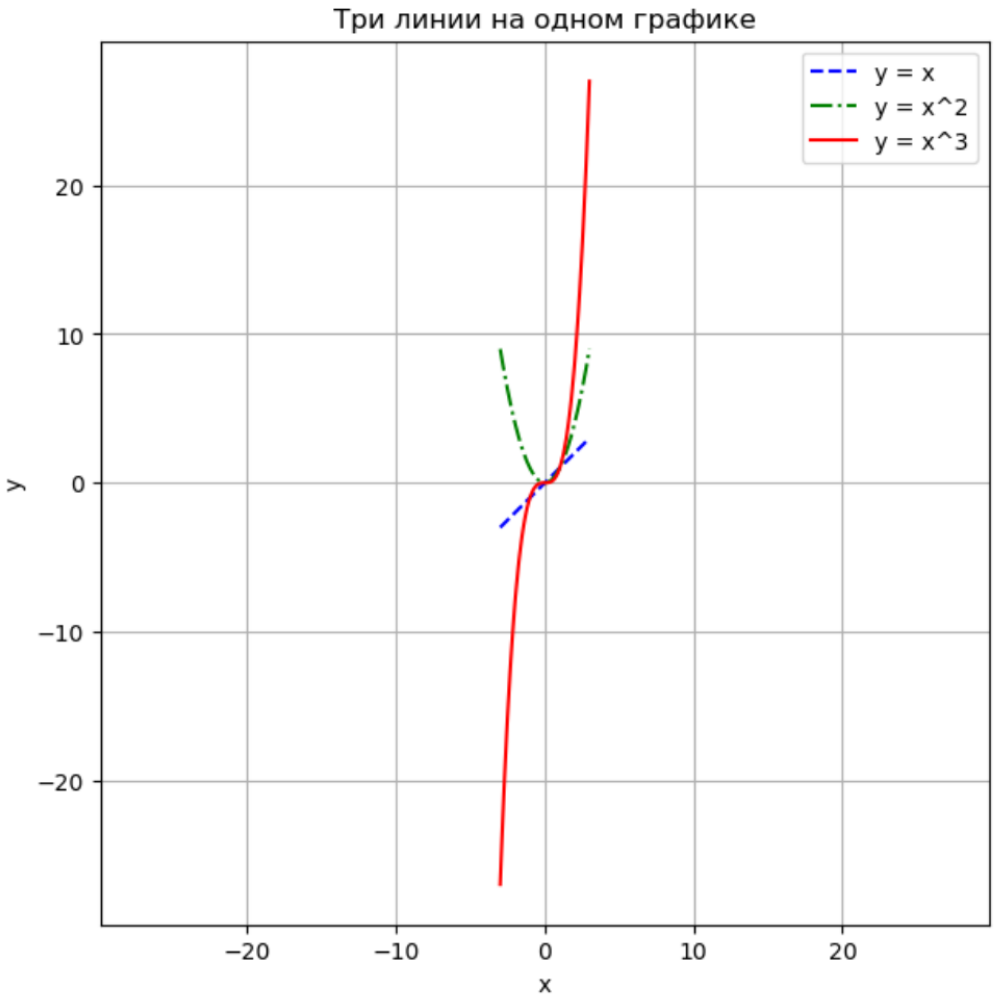

Рисунок 2. Результат выполнения задания 2

Было выполнено задание 3. С помощью библиотек NumPy и Matplotlib была визуализирована диаграмма рассеяния (scatter plot) для 50 случайных точек. В коде реализовано многомерное представление данных: положение

точки определяется координатами X и Y, цвет привязан к значению X с использованием карты viridis, а размер маркеров зависит от значения Y. Для наглядности добавлена цветовая шкала (colorbar) и настроена прозрачность элементов. Результат выполнения задания представлен на рисунке 3.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.random.rand(50)
y = np.random.rand(50)
sizes = y * 500
```

```
plt.scatter(x, y, c=x, s=sizes, cmap='viridis', alpha=0.7)
plt.title("Диаграмма рассеяния (Задание 3)")
plt.xlabel("Координата X (Цвет)")
plt.ylabel("Координата Y (Размер)")
plt.colorbar(label='Цветовая шкала X')
plt.show()
```

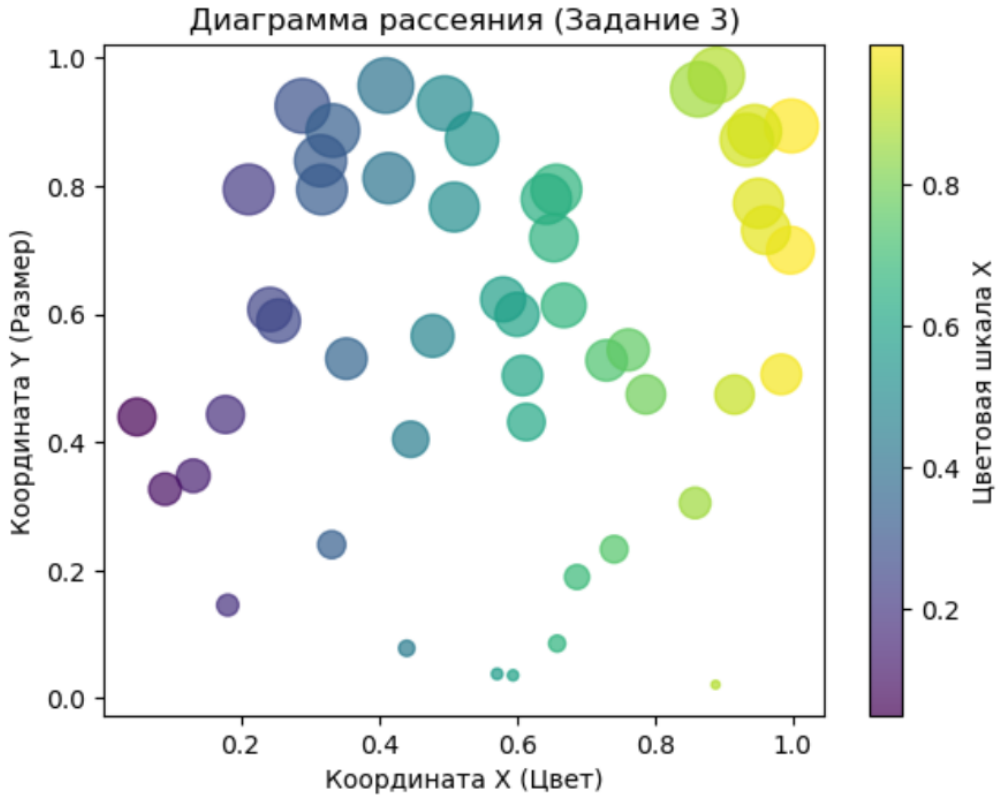

Рисунок 3. Результат выполнения задания 3

Было выполнено задание 4. С помощью библиотек NumPy и Matplotlib была построена гистограмма нормального распределения для выборки из 1000 случайных величин. В коде реализована генерация данных, расчет выборочного среднего и визуализация распределения с разбиением на 30 интервалов. На графике дополнительно отмечена линия среднего значения

(пунктиром красного цвета), добавлены легенда и подписи осей. Результат выполнения задания представлен на рисунке 4.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
data = np.random.normal(loc=0, scale=1, size=1000)
mean_val = np.mean(data)
```

```
plt.hist(data, bins=30, color='skyblue', edgecolor='black')
plt.axvline(mean_val, color='red', linestyle='dashed', linewidth=2, label=f'Среднее: {mean_val:.2f}')

plt.title("Гистограмма нормального распределения")
plt.xlabel("Значения")
plt.ylabel("Частота")
plt.legend()
plt.show()
```

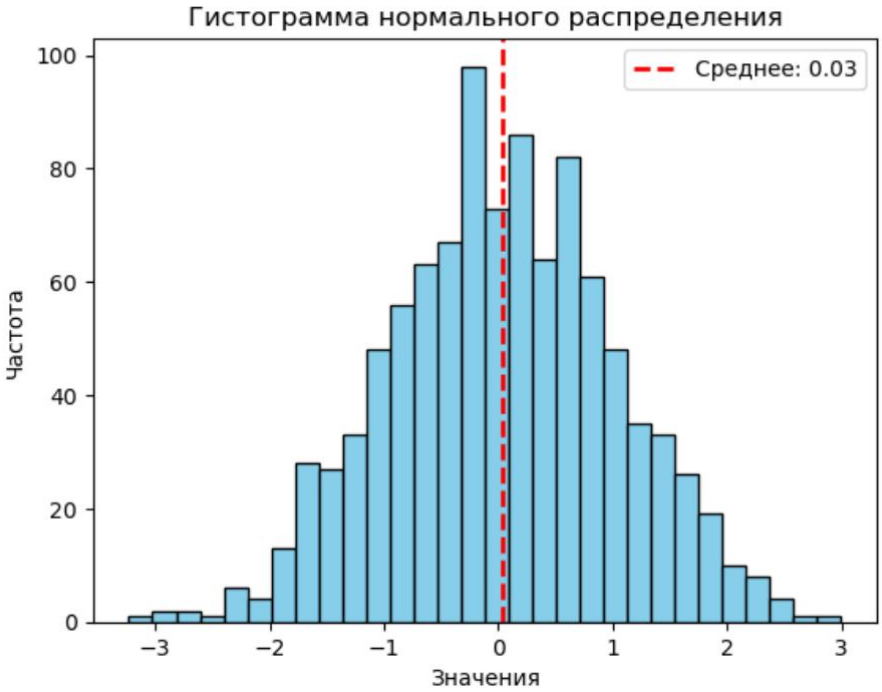

Рисунок 4. Результат выполнения задания 4

Было выполнено задание 5. С использованием библиотеки Matplotlib была построена столбчатая диаграмма, отображающая распределение оценок студентов по четырем категориям: «Отлично», «Хорошо», «Удовлетворит.» и «Неудовлетворит.». В коде заданы соответствующие количественные значения для каждой категории, настроена индивидуальная цветовая гамма для столбцов и добавлены подписи к осям и заголовок графика. Результат выполнения задания представлен на рисунке 5.

```
[1]: import matplotlib.pyplot as plt
```

```
[4]: categories = ["Отлично", "Хорошо", "Удовлетворит.", "Неудовлетворит."]
     counts = [20, 35, 30, 15]
```

```
[5]: plt.bar(categories, counts, color=['green', 'blue', 'orange', 'red'])

     plt.title("Распределение оценок студентов")
     plt.xlabel("Оценки")
     plt.ylabel("Количество студентов")

     plt.show()
```

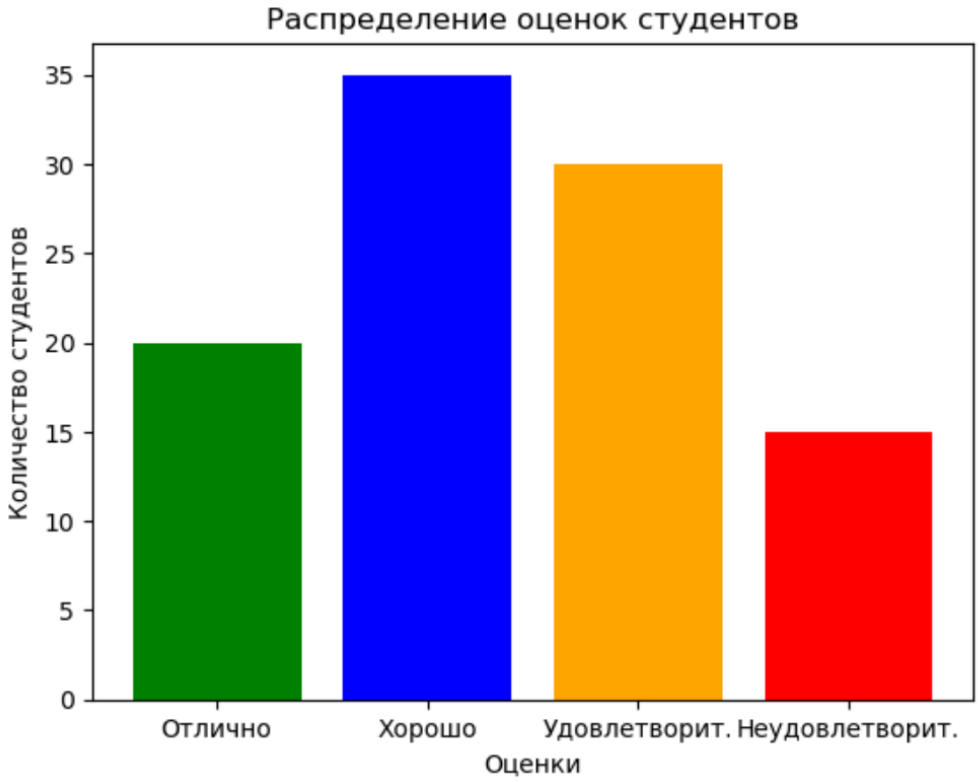

Рисунок 5. Результат выполнения задания 5

Было выполнено задание 6. С помощью библиотеки Matplotlib была построена круговая диаграмма, демонстрирующая процентное распределение оценок студентов. В коде заданы категории («Отлично», «Хорошо», «Удовлетворительно», «Неудовлетворительно») и соответствующие им значения, настроено автоматическое вычисление долей с выводом процентов на график, а также установлена цветовая палитра и размер холста. Результат выполнения задания представлен на рисунке 6.

```
[1]: import matplotlib.pyplot as plt
```

```
[2]: categories = ["Отлично", "Хорошо", "Удовлетворительно", "Неудовлетворительно"]
     counts = [20, 35, 30, 15]
```

```
[3]: plt.figure(figsize=(8, 8))
     plt.pie(counts, labels=categories, autopct='%1.1f%%', colors=['skyblue', 'lightgreen', 'or

     plt.title("Процентное распределение оценок студентов")
     plt.show()
```

Рисунок 6. Результат выполнения задания 6

Было выполнено задание 7. С использованием библиотек NumPy и Matplotlib (модуль mplot3d) был построен трехмерный график поверхности для функции z. В программном коде реализовано создание координатной сетки с помощью meshgrid, расчет значений функции и визуализация поверхности с применением цветовой карты viridis. График снабжен подписями осей X, Y, Z, заголовком и цветовой шкалой (colorbar) для наглядности. Результат выполнения задания представлен на рисунке 7.

```
[1]: import matplotlib.pyplot as plt
     import numpy as np
     from mpl_toolkits.mplot3d import Axes3D
```

```
[2]: x = np.linspace(-5, 5, 100)
     y = np.linspace(-5, 5, 100)
     x, y = np.meshgrid(x, y)
     z = np.sin(np.sqrt(x**2 + y**2))
```

```
[5]: fig = plt.figure(figsize=(10, 7))
     ax = fig.add_subplot(111, projection='3d')

     surf = ax.plot_surface(x, y, z, cmap='viridis')

     ax.set_title('3D график функции z = sin(sqrt(x^2 + y^2))')
     ax.set_xlabel('X')
     ax.set_ylabel('Y')
     ax.set_zlabel('Z')
     fig.colorbar(surf, shrink=0.5, aspect=5)

     plt.show()
```

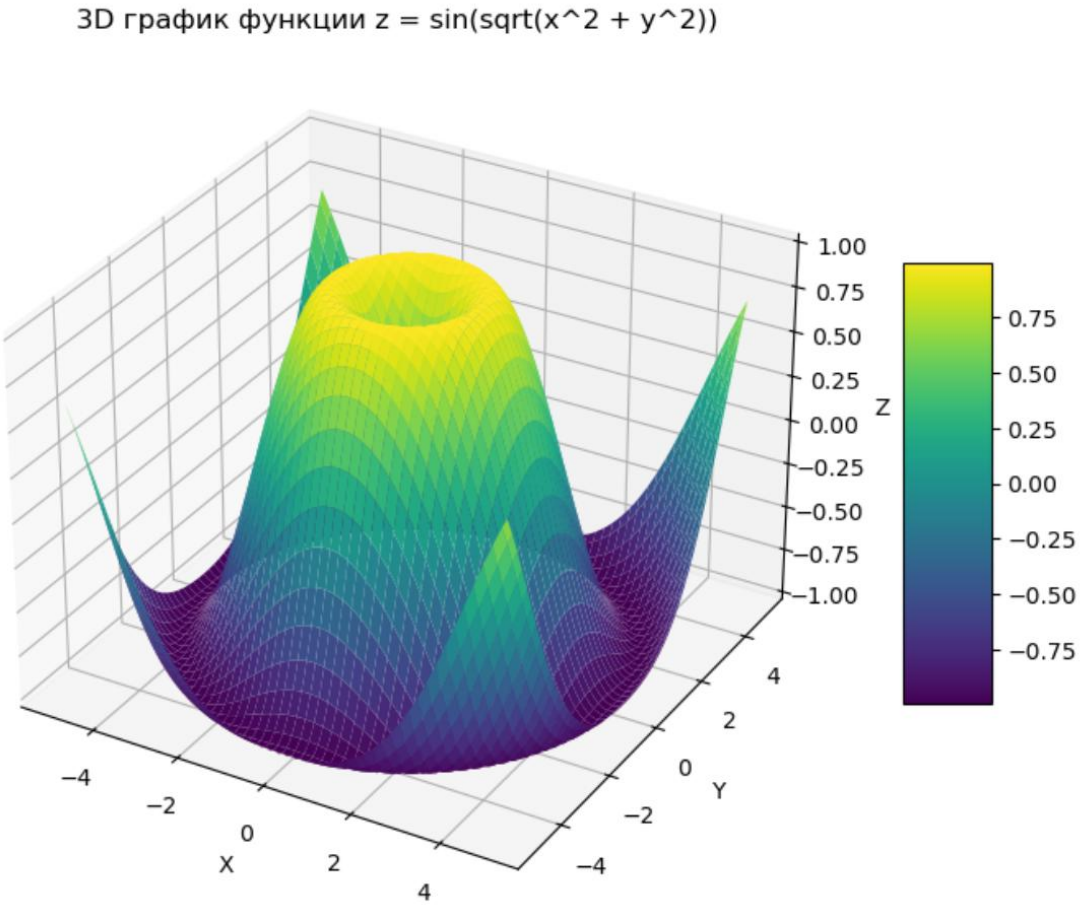

Рисунок 7. Результат выполнения задания 7

Было выполнено задание 8. С помощью библиотек NumPy и Matplotlib была реализована визуализация четырех различных графиков в одном окне с использованием сетки subplots(2, 2). В коде представлены: линейная

зависимость, парабола, а также тригонометрические функции синуса и косинуса. Для каждого подграфика задан индивидуальный заголовок, а метод tight_layout() обеспечил корректное расположение элементов без перекрытий. Результат выполнения задания представлен на рисунке 8.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(-10, 10, 100)

fig, axs = plt.subplots(2, 2, figsize=(10, 8))

axs[0, 0].plot(x, x)
axs[0, 0].set_title("Линейный график y = x")

axs[0, 1].plot(x, x**2)
axs[0, 1].set_title("Парабола y = x^2")

axs[1, 0].plot(x, np.sin(x))
axs[1, 0].set_title("Синус y = sin(x)")

axs[1, 1].plot(x, np.cos(x))
axs[1, 1].set_title("Косинус y = cos(x)")

plt.tight_layout()
plt.show()
```

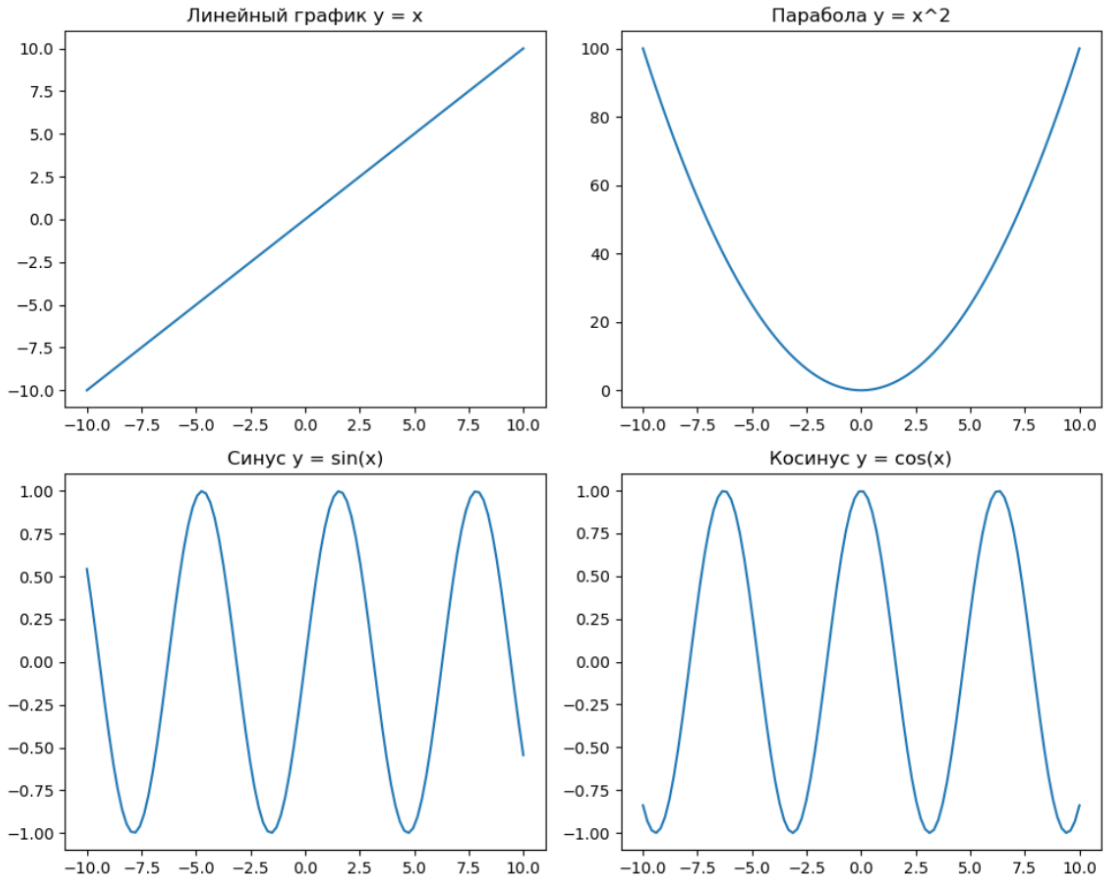

Рисунок 8. Результат выполнения задания 8

Было выполнено задание 9. С помощью библиотек NumPy и Matplotlib была построена тепловая карта (heatmap) для случайно сгенерированной матрицы размером 10x10. В коде использована функция imshow с цветовой картой viridis для визуализации интенсивности значений, добавлена цветовая шкала (colorbar) и заголовок. Результат выполнения задания представлен на рисунке 9.

```
import matplotlib.pyplot as plt
import numpy as np
```

```
matrix = np.random.rand(10, 10)

plt.imshow(matrix, cmap='viridis')
plt.colorbar()
plt.title("Тепловая карта случайной матрицы 10x10")
plt.show()
```

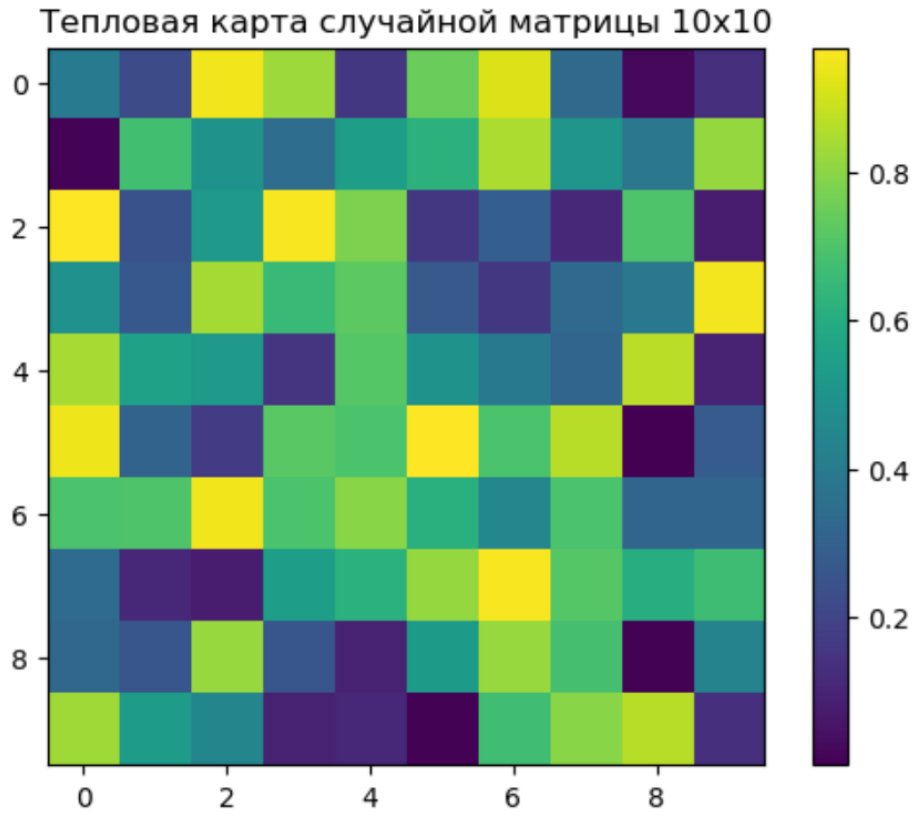

Рисунок 9. Результат выполнения задания 9

Было выполнено задание 10. С использованием библиотеки Matplotlib был построен линейный график, отображающий динамику изменения уровня воды в озере в течение года. В коде задан список месяцев и соответствующие им значения уровня воды, а для визуализации использована синяя линия с круглыми маркерами в точках данных. График дополнен заголовком,

информативными подписями осей и координатной сеткой для точного считывания значений. Результат выполнения задания представлен на рисунке 10.

```
[1]: import matplotlib.pyplot as plt

[2]: months = ['Янв', 'Фев', 'Март', 'Апр', 'Май', 'Июнь', 'Июль', 'Авг', 'Сен', 'Окт', 'Нояб',
     water_level = [2.3, 2.4, 2.6, 2.8, 3.0, 3.1, 3.2, 3.1, 3.0, 2.8, 2.6, 2.4]

[3]: plt.plot(months, water_level, color='blue', marker='o')

     plt.title("Уровень воды в озере в течение года")
     plt.xlabel("Месяцы")
     plt.ylabel("Уровень воды (м)")
     plt.grid(True)

     plt.show()
```

Рисунок 10. Результат выполнения задания 10

Было выполнено задание 11. С помощью библиотеки Matplotlib была построена столбчатая диаграмма, визуализирующая среднюю температуру по четырем сезонам. В коде реализована привязка каждого времени года к определенному цвету (синий для зимы, красный для лета и т.д.), добавлены информативные подписи осей и заголовок. Также настроена пунктирная сетка

по оси Y для более удобного сопоставления высоты столбцов со значениями температуры. Результат выполнения задания представлен на рисунке 11.

```
import matplotlib.pyplot as plt
```

```
seasons = ['Зима', 'Весна', 'Лето', 'Осень']
temperatures = [-5, 10, 25, 12]
colors = ['blue', 'lightgreen', 'red', 'orange']
```

```
plt.bar(seasons, temperatures, color=colors)

plt.title("Средняя температура по сезонам")
plt.xlabel("Сезоны")
plt.ylabel("Температура (C)")
plt.grid(axis='y', linestyle='--', alpha=0.7)

plt.show()
```

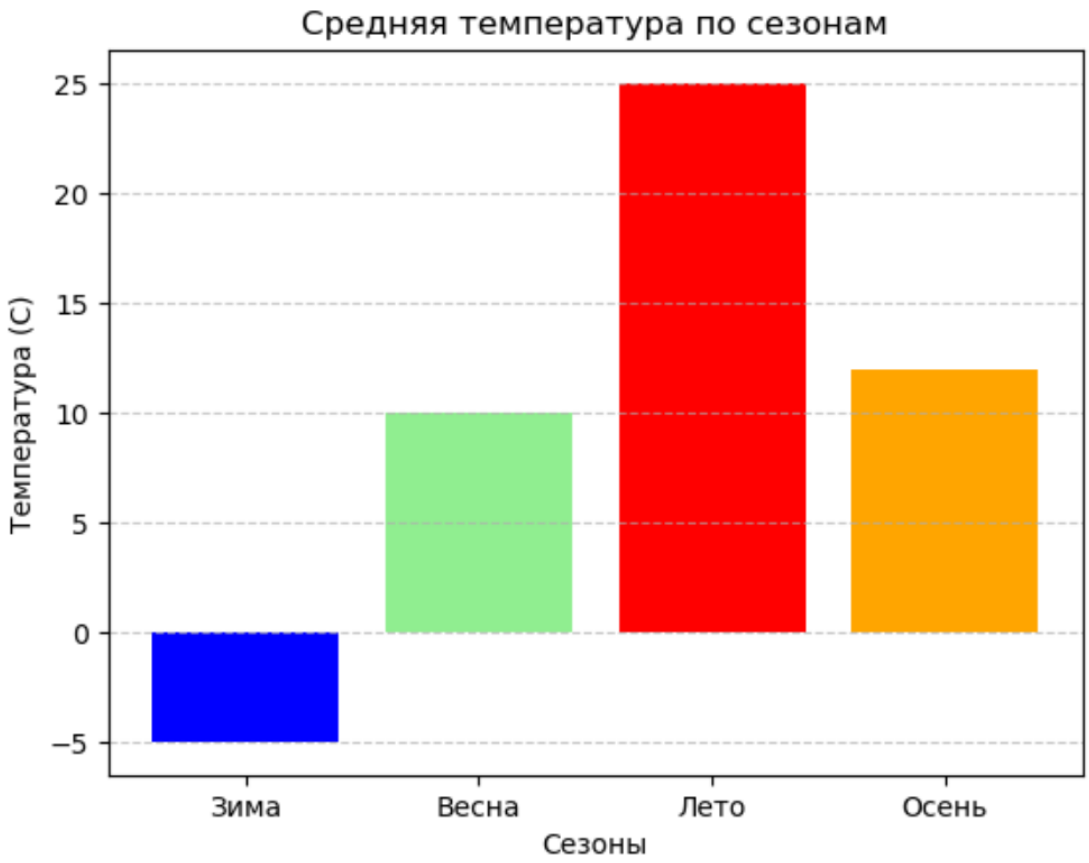

Рисунок 11. Результат выполнения задания 11

Было выполнено задание 12. С использованием библиотек NumPy, Matplotlib и SciPy был произведен расчет и визуализация определенного интеграла функции f(x) = 1/x на интервале от 1 до 5. В коде реализовано численное интегрирование методом quad, построение графика функции и выделение площади под кривой с помощью заливки. Для наглядности на

график выведены оси координат, легенда с результатом вычислений и пунктирная сетка. Результат выполнения задания представлен на рисунке 12.

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import quad
```

```
def f(x):
    return 1/x
```

```
x_full = np.linspace(0.5, 6, 400)
x_int = np.linspace(1, 5, 100)

area, error = quad(f, 1, 5)
```

```
plt.figure(figsize=(8, 6))
plt.plot(x_full, f(x_full), 'b', label='f(x) = 1/x')
plt.fill_between(x_int, f(x_int), color='skyblue', alpha=0.5, label=f'Площадь ≈ {area:.3f}')

plt.axhline(0, color='black', linewidth=1)
plt.axvline(0, color='black', linewidth=1)

plt.title("Визуализация определенного интеграла f(x) = 1/x")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.grid(True, linestyle=':')

plt.show()
```

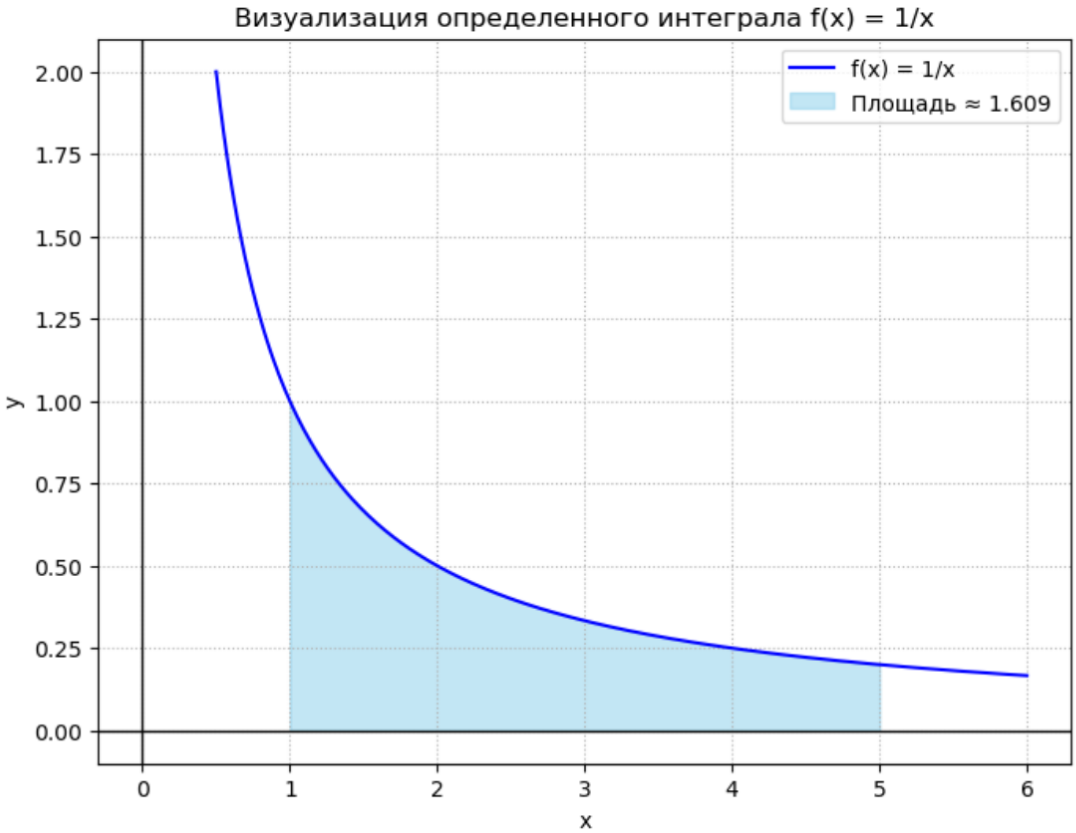

Рисунок 12. Результат выполнения задания 12

Было выполнено задание 13. С помощью библиотек NumPy и Matplotlib (модуль mplot3d) была реализована сравнительная визуализация трехмерного графика функции z. В коде продемонстрированы два способа отображения поверхности: в виде плотной заливки с цветовой картой viridis (plot_surface) и в виде каркасной сетки синего цвета (plot_wireframe). Для обоих графиков настроены подписи осей, индивидуальные заголовки и общая компоновка, а для первого варианта дополнительно выведена цветовая шкала. Результат выполнения задания представлен на рисунке 13.

```
import matplotlib.pyplot as plt
import numpy as np
from mpl_toolkits.mplot3d import Axes3D
```

```
x = np.linspace(-5, 5, 50)
y = np.linspace(-5, 5, 50)
x, y = np.meshgrid(x, y)
z = np.cos(np.sqrt(x**2 + y**2))
```

```
fig = plt.figure(figsize=(14, 6))

ax1 = fig.add_subplot(121, projection='3d')
surf = ax1.plot_surface(x, y, z, cmap='viridis')
ax1.set_title('Плотная поверхность (plot_surface)')
ax1.set_xlabel('X')
ax1.set_ylabel('Y')
ax1.set_zlabel('Z')
fig.colorbar(surf, ax=ax1, shrink=0.5, aspect=10)

ax2 = fig.add_subplot(122, projection='3d')
ax2.plot_wireframe(x, y, z, color='blue', linewidth=0.5)
ax2.set_title('Каркасная модель (plot_wireframe)')
ax2.set_xlabel('X')
ax2.set_ylabel('Y')
ax2.set_zlabel('Z')

plt.tight_layout()
plt.show()
```

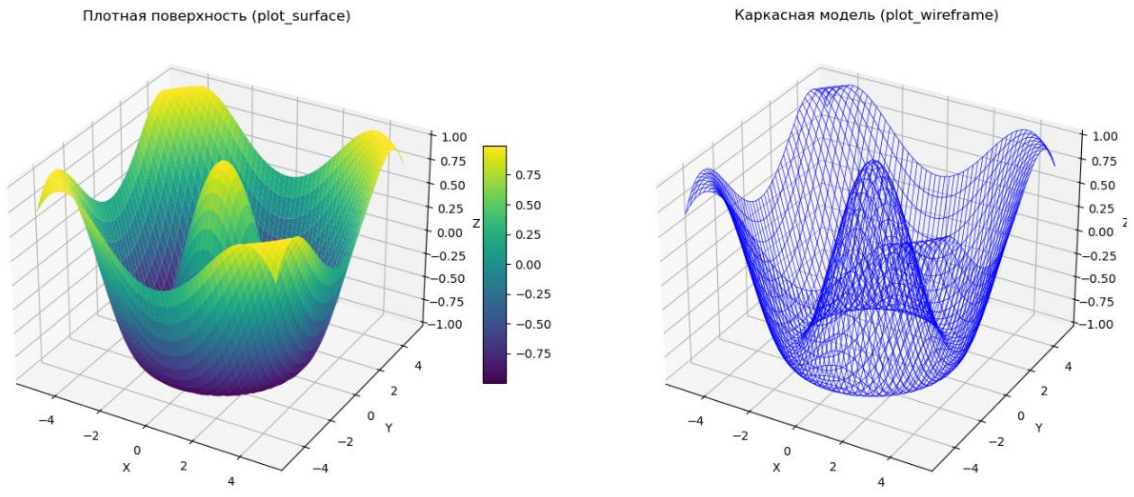

Рисунок 13. Результат выполнения задания 13

Все изменения были зафиксированы и отправлены на сервер GitHub.

Контрольные вопросы:

1. Установка пакета matplotlib выполняется командой pip install matplotlib.
2. "Магическая" команда: %matplotlib inline (для отображения графиков прямо в ноутбуке).
3. Отобразить график с помощью функции plot можно, вызвав plt.plot(x, y) и затем plt.show().
4. Отобразить несколько графиков на одном поле можно, вызвав функцию plot() несколько раз перед вызовом show().
5. Метод для построения диаграмм категориальных данных – это чаще всего plt.bar() для столбчатых или plt.pie() для круговых диаграмм.
6. Основные элементы графика – это Figure (окно), Axes (область рисования), Axis (оси), Title (заголовок) и Legend (легенда).
7. Управление текстовыми надписями осуществляется функциями plt.title(), plt.xlabel(), plt.ylabel() и plt.text().
8. Управление легендой графика выполняется через аргумент label в функциях построения и вызов plt.legend().
9. Задать цвет и стиль линий можно с помощью аргументов color и linestyle в функции plot().
10. Размещение графика в разных полях выполняется с помощью функций plt.subplot() или plt.subplots().
11. Построение линейного графика осуществляется вызовом основной функции plt.plot().
12. Заливка области между графиком и осями (или двумя графиками) выполняется методом plt.fill_between().
13. Выборочная заливка по условию реализуется с помощью параметра where в методе fill_between().
14. Двухцветная заливка выполняется путем двух последовательных вызовов fill_between() с разными условиями и цветами.

15. Маркировка графиков задается аргументом marker (например, 'o' или 's') в функции plot().

16. Обрезка графиков (установка границ осей) осуществляется функциями plt.xlim() и plt.ylim().

17. Ступенчатый график строится функцией plt.step(), его особенность – отображение данных в виде ломаной с резкими переходами (ступенями).

18. Стековый график строится функцией plt.stackplot(), он показывает, как несколько наборов данных складываются друг на друга, формируя общую сумму.

19. Stem-график строится функцией plt.stem(), его особенность в визуализации дискретных значений в виде вертикальных линий с маркерами на концах.

20. Точечный график строится функцией plt.scatter(), его особенность – возможность независимо управлять цветом и размером каждой отдельной точки.

21. Построение столбчатых диаграмм осуществляется методами plt.bar() (вертикальные) и plt.barh() (горизонтальные).

22. Групповая диаграмма — это несколько столбцов рядом для разных категорий; errorbar – это линии погрешности, добавленные к столбцам.

23. Построение круговой диаграммы выполняется с помощью функции plt.pie().

24. Цветовая карта (colormap) – это схема соответствия чисел цветам; работа с ними идет через аргумент cmap (например, cmap='viridis').

25. Отображение изображения осуществляется методом plt.imshow().

26. Отображение тепловой карты выполняется также методом plt.imshow() или plt.pcolormesh() на основе матрицы данных.

27. Линейный 3D-график строится методом ax.plot() на осях с проекцией projection='3d'.

28. Точечный 3D-график создается методом ax.scatter() в трехмерном пространстве.

29. Построение каркасной поверхности выполняется методом ax.plot_wireframe().

30. Построение трехмерной поверхности осуществляется методом ax.plot_surface().

Вывод: В результате работы были исследованы базовые возможности библиотеки matplotlib языка программирования Python.